2022 World Cup: An Event-Level Football Analytics Using StatsBomb Open Data

Introduction

The 2022 FIFA World Cup in Qatar brought together 32 national teams competing at the highest level of international football, generating rich event-level data that captures tactical decisions, player actions, and match dynamics.

This project analyzes the tournament using StatsBomb Open Data, applying modern data science and football analytics techniques to study passes, shots, defensive actions, and team structures. Through pitch-based visualizations built with mplsoccer, the analysis transforms complex football data into clear, interpretable insights aligned with professional broadcast and club-level analysis standards.

Aims & Objectives

Primary Aim:

To conduct a comprehensive, data-driven analysis of the 2022 FIFA World Cup using event-level data, revealing tactical patterns, performance metrics, and strategic insights across both tournament-wide and match-specific contexts.

Objectives:

- Analyze tactical formations, expected goals (xG) accumulation, shot quality and player positioning between finalists, Argentina and France
- Identify top-performing teams, players across key metrics (goals, xG, possession, assist, passing).
- Determine optimal scoring periods and vulnerability windows across matches.
- Evaluate set-piece effectiveness vs open play goal distribution.
- Create broadcast-quality visualizations insights matching professional analytics standards.

Load 2022 World Cup Data

In [58]:
```
# Import required libraries

from statsbombpy import sb
```

```
import pandas as pd
import numpy as np
from mplsoccer import Pitch
import matplotlib.pyplot as plt
```

In [ ]:

```
```

Get competitions

In [4]:

```
competitions = sb.competitions()
```

Filter World Cup 2022

In [5]:

```
wc_2022 = competitions[
    (competitions["competition_name"] == "FIFA World Cup") &
    (competitions["season_name"] == "2022")
]
```

Load matches

In [6]:

```
matches = sb.matches(
    competition_id=wc_2022.iloc[0]["competition_id"],
    season_id=wc_2022.iloc[0]["season_id"]
)
```

Pick one match:

In [7]:

```
match_id = matches.iloc[0]["match_id"]
```

Load Event Level Data

In [8]:
```
events = sb.events(match_id=match_id)
```

In [ ]:
```

```

Argentina VS France Final Match Analysis

In [9]:
```
# Load Argentina vs France final
argentina_match_id = 3869685  # The World Cup final!
```

In [10]:
```
# Load events for that match
events = sb.events(match_id=argentina_match_id)
```

In [11]:
```
# Filter for Argentina events only
argentina_events = events[events['team'] == 'Argentina'].copy()
```

In [12]:
```
# Calculate average positions per player
# Only include events that have location data
argentina_with_location = argentina_events[argentina_events['location'].notna()]

avg_positions = argentina_with_location.groupby('player').agg({
    'location': list
}).reset_index()
```

In [13]:
```
# Calculate average x and y positions for Argentina

avg_positions['avg_x'] = avg_positions['location'].apply(
```

```
    lambda locs: sum([loc[0] for loc in locs]) / len(locs) if locs else 0
)
avg_positions['avg_y'] = avg_positions['location'].apply(
    lambda locs: sum([loc[1] for loc in locs]) / len(locs) if locs else 0
)
avg_positions['actions'] = avg_positions['location'].apply(len)
avg_positions['team'] = 'Argentina'
```

In [14]:
```
# filter for Argentina
argentina = avg_positions[avg_positions["team"] == "Argentina"].copy()
```

In [15]:
```
print(f"Found {len(argentina)} Argentina players")
print(argentina[['player', 'avg_x', 'avg_y', 'actions']].head(10))
```

```
Found 17 Argentina players
                          player      avg_x      avg_y  actions
0             Alexis Mac Allister  65.755897  24.585128      195
1          Cristian Gabriel Romero  33.388670  51.943842      203
2        Damián Emiliano Martínez    9.812346  42.340741       81
3                  Enzo Fernandez  53.426220  43.403354      328
4       Germán Alejandro Pezzella  53.300000  30.666667        3
5           Gonzalo Ariel Montiel  60.761818  68.527273       55
6                  Julián Álvarez  76.200680  40.263265      147
7          Lautaro Javier Martínez  93.835294  35.997059       34
8           Leandro Daniel Paredes  51.535556  42.988889       45
9  Lionel Andrés Messi Cuccittini  76.269038  51.536820      239
```

In [16]:
```
# Fix Messi's name display
argentina.loc[argentina['player'].str.contains('Messi', case=False), 'display_na
```

In [17]:
```
# For all other players, use last name
argentina.loc[argentina['display_name'].isna(), 'display_name'] = argentina.loc[
```

In [ ]:
```

```

In [18]:
```
# Filter for France events from the same match
france_events = events[events['team'] == 'France'].copy()
```

In [19]:
```
# Calculate average positions per player
# Only include events that have location data
france_with_location = france_events[france_events['location'].notna()].copy()

avg_positions_france = france_with_location.groupby('player').agg({
    'location': list
}).reset_index()
```

In [20]:
```
# Calculate average x and y positions for France

avg_positions_france['avg_x'] = avg_positions_france['location'].apply(
    lambda locs: sum([loc[0] for loc in locs]) / len(locs) if locs else 0
)
avg_positions_france['avg_y'] = avg_positions_france['location'].apply(
    lambda locs: sum([loc[1] for loc in locs]) / len(locs) if locs else 0
)
avg_positions_france['actions'] = avg_positions_france['location'].apply(len)
avg_positions_france['team'] = 'France'
```

In [21]:
```
# Filter for France
france = avg_positions_france[avg_positions_france["team"] == "France"].copy()
```

```
print(f"Found {len(france)} France players")
print(france[['player', 'avg_x', 'avg_y', 'actions']])
```

```
Found 17 France players
                            player      avg_x      avg_y  actions
0                     Adrien Rabiot  62.047126  26.544828      174
1                  Antoine Griezmann  63.973529  43.860784      102
2          Aurélien Djani Tchouaméni  60.119512  39.119024      205
3           Dayotchanculle Upamecano  44.557576  26.196465      198
4                  Eduardo Camavinga  49.175904  11.893976       83
5                       Hugo Lloris  10.190625  38.872917       96
6                   Ibrahima Konaté  64.700000  54.172727       11
7                      Jules Koundé  51.271429  67.521905      210
8                    Kingsley Coman  65.895294  57.614118       85
9              Kylian Mbappé Lottin  83.687919  18.302685      149
10                   Marcus Thuram  68.993277  17.086555      119
11                   Olivier Giroud  72.702941  43.985294       34
12                  Ousmane Dembélé  65.325490  69.470588       51
13                Randal Kolo Muani  78.645669  49.825197      127
14                   Raphaël Varane  43.250829  50.269061      181
15  Theo Bernard François Hernández  62.384615   8.016084      143
16                  Youssouf Fofana  66.204878  26.800000       41
```

In [22]:
```
# Fix special player names if needed (e.g., Mbappé)

france['display_name'] = france['player'].apply(lambda x: x.split()[-1])
```

In [23]:
```
france.loc[france['player'].str.contains('Mbappé', case=False), 'display_name']
```

In [ ]:
```

```

Visualizing Average Player Positions : Argentina VS France Final

In [24]:

```
from mplsoccer import Pitch
import matplotlib.pyplot as plt

# Create side-by-side pitches
pitch = Pitch(
    pitch_type="statsbomb",
    pitch_color="#1b7a3a",
    line_color="white"
)

fig, axes = plt.subplots(1, 2, figsize=(16, 7))

# ==== ARGENTINA (Left) ====
pitch.draw(ax=axes[0])

argentina["size"] = argentina["actions"] * 25
argentina.loc[argentina["size"] < 200, "size"] = 200

pitch.scatter(
    argentina["avg_x"],
    argentina["avg_y"],
    s=argentina["size"],
    color="#fde68a",
    edgecolors="black",
    linewidth=1.2,
    alpha=0.9,
    ax=axes[0]
```

```
)

for _, row in argentina.iterrows():
    axes[0].text(
        row["avg_x"],
        row["avg_y"],
        row["display_name"],
        ha="center",
        va="center",
        fontsize=9,
        weight="bold"
    )

axes[0].set_title(
    "Argentina – Average Player Positions\n(Size = On-ball Involvement)",
    fontsize=12,
    weight="bold"
)

# ==== FRANCE (Right) ====
pitch.draw(ax=axes[1])

france["size"] = france["actions"] * 25
france.loc[france["size"] < 200, "size"] = 200

pitch.scatter(
    france["avg_x"],
    france["avg_y"],
    s=france["size"],
    color="#4169E1",
    edgecolors="white",
    linewidth=1.2,
```

```
    alpha=0.9,
    ax=axes[1]
)

for _, row in france.iterrows():
    axes[1].text(
        row["avg_x"],
        row["avg_y"],
        row["display_name"],
        ha="center",
        va="center",
        fontsize=9,
        weight="bold",
        color="white"
    )

axes[1].set_title(
    "France – Average Player Positions\n(Size = On-ball Involvement)",
    fontsize=12,
    weight="bold"
)

# Overall title
fig.suptitle(
    "2022 World Cup Final: Argentina vs France\nAverage Player Positions Compari
    fontsize=16,
    weight="bold",
    y=0.98
)

plt.tight_layout()
plt.show()
```

Insights:

Argentina:

- Argentina maintained a balanced 4-3-3 formation with Messi operating high and wide on the right flank, indicating an attacking setup focused on utilizing his creativity.
- The midfield trio (Fernández, Mac Allister, De Paul) positioned centrally shows Argentina's emphasis on controlling the middle of the pitch and building attacks through the center.

France:

- France adopted a more compact 4-2-3-1 formation with Mbappé positioned high and wide, mirroring Messi's role but on the opposite flank.
- The deeper positioning of Tchouaméni and Rabiot as a double pivot indicates France's focus on defensive stability and counter-attacking opportunities.

In [ ]:

Pass Networks - Argentina VS France

What Were the Key Differences in Passing Patterns Between Argentina and France?

In [25]:
```
# Analyzing Pass Networks
def get_pass_network(events_df, team_name, min_passes=3):
    """Calculate pass network for a team"""
    team_events = events_df[(events_df['team'] == team_name) & (events_df['locat

    # Average positions
    positions = team_events.groupby('player')['location'].apply(
        lambda x: [np.mean([loc[0] for loc in x]), np.mean([loc[1] for loc in x]
    ).reset_index()
    positions[['avg_x', 'avg_y', 'num_events']] = pd.DataFrame(positions['locati

    # Pass connections
    passes = events_df[(events_df['team'] == team_name) & (events_df['type'] ==
    connections = passes.groupby(['player', 'pass_recipient']).size().reset_inde
    connections = connections[connections['count'] >= min_passes]
```

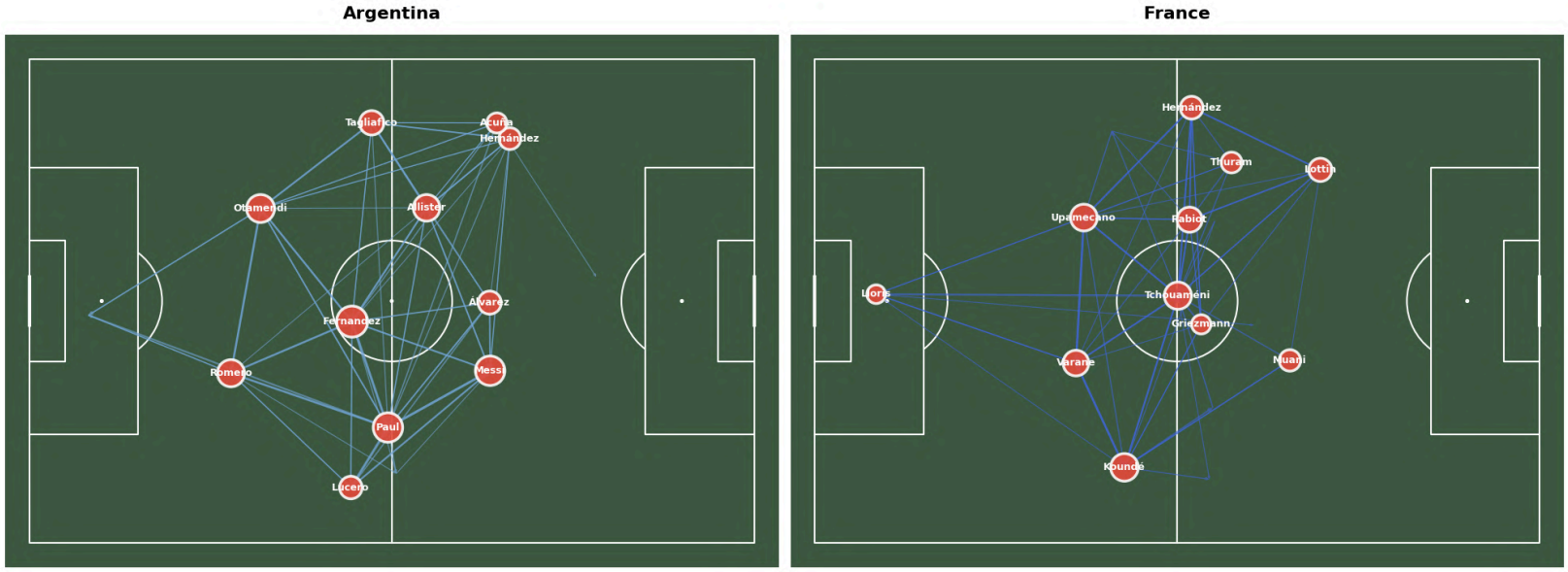
```
    # Merge positions
    connections = connections.merge(positions[['player', 'avg_x', 'avg_y']], on=
    connections = connections.merge(positions[['player', 'avg_x', 'avg_y']],
                                    left_on='pass_recipient', right_on='player',
                                    suffixes=('_from', '_to'))

    return positions.nlargest(11, 'num_events'), connections

argentina_pos, argentina_conn = get_pass_network(events, 'Argentina')
france_pos, france_conn = get_pass_network(events, 'France')
```

In [26]:

```
# Visualize Side-by-Side

pitch = Pitch(pitch_type="statsbomb", pitch_color="#3d5941", line_color="white",
fig, axes = plt.subplots(1, 2, figsize=(20, 10), facecolor='white')
fig.suptitle('Pass Networks - 2022 World Cup Final Argentina VS France', fontsiz

for ax, positions, connections, team, color in zip(
    axes,
    [argentina_pos, france_pos],
    [argentina_conn, france_conn],
    ['Argentina', 'France'],
    ['#75AADB', '#4169E1']
):
    pitch.draw(ax=ax)

    # Pass arrows
    for _, row in connections.iterrows():
        pitch.arrows(row['avg_x_from'], row['avg_y_from'], row['avg_x_to'], row[
                     width=np.sqrt(row['count'])*0.5, headwidth=3, headlength=3,
                     color=color, alpha=0.6, ax=ax, zorder=1)
```

```
    # Player nodes
    node_sizes = np.clip(positions['num_events']*3, 200, 800)
    pitch.scatter(positions['avg_x'], positions['avg_y'], s=node_sizes,
                  color='#E74C3C', edgecolors='white', linewidth=2.5,
                  alpha=0.9, ax=ax, zorder=3)

    # Labels
    for _, player in positions.iterrows():
        label = 'Messi' if 'Messi' in player['player'] else player['player'].spl
        ax.text(player['avg_x'], player['avg_y'], label, fontsize=9, weight='bol
                ha='center', va='center', color='white', zorder=4)

    ax.set_title(team, fontsize=16, weight='bold', pad=15)

fig.text(0.5, 0.02, "Arrow width = Pass frequency | Node size = Player involveme
         ha='center', fontsize=11, style='italic')

plt.tight_layout(rect=[0, 0.03, 1, 0.96])
plt.show()
```

Pass Networks - 2022 World Cup Final Argentina VS France



Arrow width = Pass frequency | Node size = Player involvement

Key Insights:

- Argentina’s pass network is dense and well-connected through central midfield, showing sustained possession.
- The French's pass network appears more dispersed with fewer connections, suggesting a more direct, counter-attacking approach rather than sustained possession-based play.
- Griezmann acted as the central pivot connecting defense to attack through direct vertical passes.

In [ ]:

Key Player Analysis

Messi and Mbappe

In [27]:
```
# Extract Messi and Mbappé Data

# Filter shots for both players
messi_shots = events[
    (events['player'].str.contains('Messi', case=False, na=False)) &
    (events['type'] == 'Shot')
].copy()

mbappe_shots = events[
    (events['player'].str.contains('Mbappé', case=False, na=False)) &
    (events['type'] == 'Shot')
].copy()
```

In [28]:
```
# Extract location data

messi_shots['x'] = messi_shots['location'].str[0]
messi_shots['y'] = messi_shots['location'].str[1]
messi_shots['xg'] = messi_shots['shot_statsbomb_xg']
messi_shots['goal'] = messi_shots['shot_outcome'] == 'Goal'

mbappe_shots['x'] = mbappe_shots['location'].str[0]
mbappe_shots['y'] = mbappe_shots['location'].str[1]
```

```
mbappe_shots['xg'] = mbappe_shots['shot_statsbomb_xg']
mbappe_shots['goal'] = mbappe_shots['shot_outcome'] == 'Goal'
```

In [29]:
```
# Calculate Key Metrics - Messi and Mbappe

# Messi metrics
messi_total_shots = len(messi_shots)
messi_total_xg = messi_shots['xg'].sum()
messi_xg_per_shot = messi_total_xg / messi_total_shots if messi_total_shots > 0
messi_goals = messi_shots['goal'].sum()

# Mbappé metrics
mbappe_total_shots = len(mbappe_shots)
mbappe_total_xg = mbappe_shots['xg'].sum()
mbappe_xg_per_shot = mbappe_total_xg / mbappe_total_shots if mbappe_total_shots
mbappe_goals = mbappe_shots['goal'].sum()
```

In [30]:
```
# Assists (that's pass events that led to goals)
messi_assists = events[
    (events['player'].str.contains('Messi', case=False, na=False)) &
    (events['pass_goal_assist'] == True)
].shape[0]

mbappe_assists = events[
    (events['player'].str.contains('Mbappé', case=False, na=False)) &
    (events['pass_goal_assist'] == True)
].shape[0]

print("=" * 50)
print("MESSI vs MBAPPÉ - 2022 WORLD CUP FINAL")
print("=" * 50)
```

```
print(f"\nLionel Messi:")
print(f"  Shots: {messi_total_shots}")
print(f"  Total xG: {messi_total_xg:.2f}")
print(f"  xG per shot: {messi_xg_per_shot:.3f}")
print(f"  Goals: {int(messi_goals)}")
print(f"  Assists: {messi_assists}")
print(f"  Goal Involvement: {int(messi_goals) + messi_assists}")

print(f"\nKylian Mbappé:")
print(f"  Shots: {mbappe_total_shots}")
print(f"  Total xG: {mbappe_total_xg:.2f}")
print(f"  xG per shot: {mbappe_xg_per_shot:.3f}")
print(f"  Goals: {int(mbappe_goals)}")
print(f"  Assists: {mbappe_assists}")
print(f"  Goal Involvement: {int(mbappe_goals) + mbappe_assists}")
print("=" * 50)
```

```
==================================================
MESSI vs MBAPPÉ - 2022 WORLD CUP FINAL
==================================================

Lionel Messi:
  Shots: 6
  Total xG: 2.24
  xG per shot: 0.373
  Goals: 3
  Assists: 0
  Goal Involvement: 3

Kylian Mbappé:
  Shots: 7
  Total xG: 2.57
  xG per shot: 0.367
  Goals: 4
  Assists: 0
  Goal Involvement: 4
==================================================
```

In [31]:

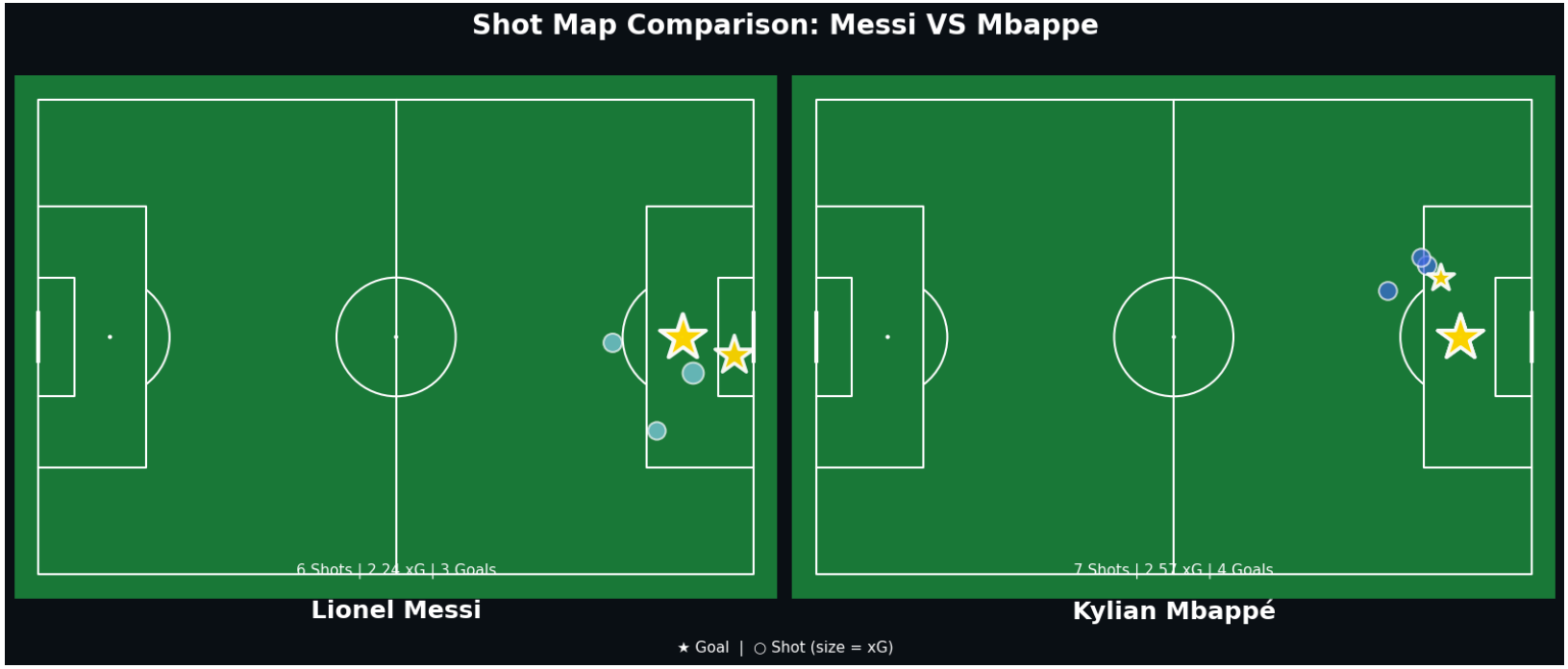
```python
# VISUALIZATION : Side-by-Side Shot Maps

pitch = Pitch(
    pitch_type="statsbomb",
    pitch_color="#1b7a3a",
    line_color="white",
    linewidth=1.5
)

fig, axes = plt.subplots(1, 2, figsize=(16, 7), facecolor='#0e1117')
```

```python
# MESSI Shot Map
pitch.draw(ax=axes[0])

# Plot shots
for _, shot in messi_shots.iterrows():
    if shot['goal']:
        # Goals - larger, gold with star
        pitch.scatter(
            shot['x'], shot['y'],
            s=shot['xg'] * 1200 + 300,
            color='#FFD700',
            edgecolors='white',
            linewidth=2.5,
            alpha=0.95,
            ax=axes[0],
            zorder=3,
            marker='*'
        )
    else:
        # Non-goals
        pitch.scatter(
            shot['x'], shot['y'],
            s=shot['xg'] * 800 + 150,
            color='#87CEEB',
            edgecolors='white',
            linewidth=1.5,
            alpha=0.7,
            ax=axes[0],
            zorder=2
        )

axes[0].text(
```

```
    60, 84, 'Lionel Messi',
    fontsize=18, weight='bold', color='white',
    ha='center', va='top'
)
axes[0].text(
    60, 78,
    f'{messi_total_shots} Shots | {messi_total_xg:.2f} xG | {int(messi_goals)} G
    fontsize=11, color='white', ha='center', va='top'
)

# MBAPPÉ Shot Map
pitch.draw(ax=axes[1])

# Plot shots
for _, shot in mbappe_shots.iterrows():
    if shot['goal']:
        # Goals - larger, gold with star
        pitch.scatter(
            shot['x'], shot['y'],
            s=shot['xg'] * 1200 + 300,
            color='#FFD700',
            edgecolors='white',
            linewidth=2.5,
            alpha=0.95,
            ax=axes[1],
            zorder=3,
            marker='*'
        )
    else:
        # Non-goals
        pitch.scatter(
            shot['x'], shot['y'],
```

```
            s=shot['xg'] * 800 + 150,
            color='#4169E1',
            edgecolors='white',
            linewidth=1.5,
            alpha=0.7,
            ax=axes[1],
            zorder=2
        )

axes[1].text(
    60, 84, 'Kylian Mbappé',
    fontsize=18, weight='bold', color='white',
    ha='center', va='top'
)
axes[1].text(
    60, 78,
    f'{mbappe_total_shots} Shots | {mbappe_total_xg:.2f} xG | {int(mbappe_goals)}
    fontsize=11, color='white', ha='center', va='top'
)

# Overall title
fig.text(
    0.5, 0.96,
    'Shot Map Comparison: Messi VS Mbappe',
    fontsize=20, weight='bold', color='white',
    ha='center', va='top'
)

# Legend
fig.text(
    0.5, 0.02,
    '★ Goal  |  ○ Shot (size = xG)',
```

```
    fontsize=11, color='white',
    ha='center', va='bottom'
)

plt.tight_layout(rect=[0, 0.03, 1, 0.94])
plt.show()
```

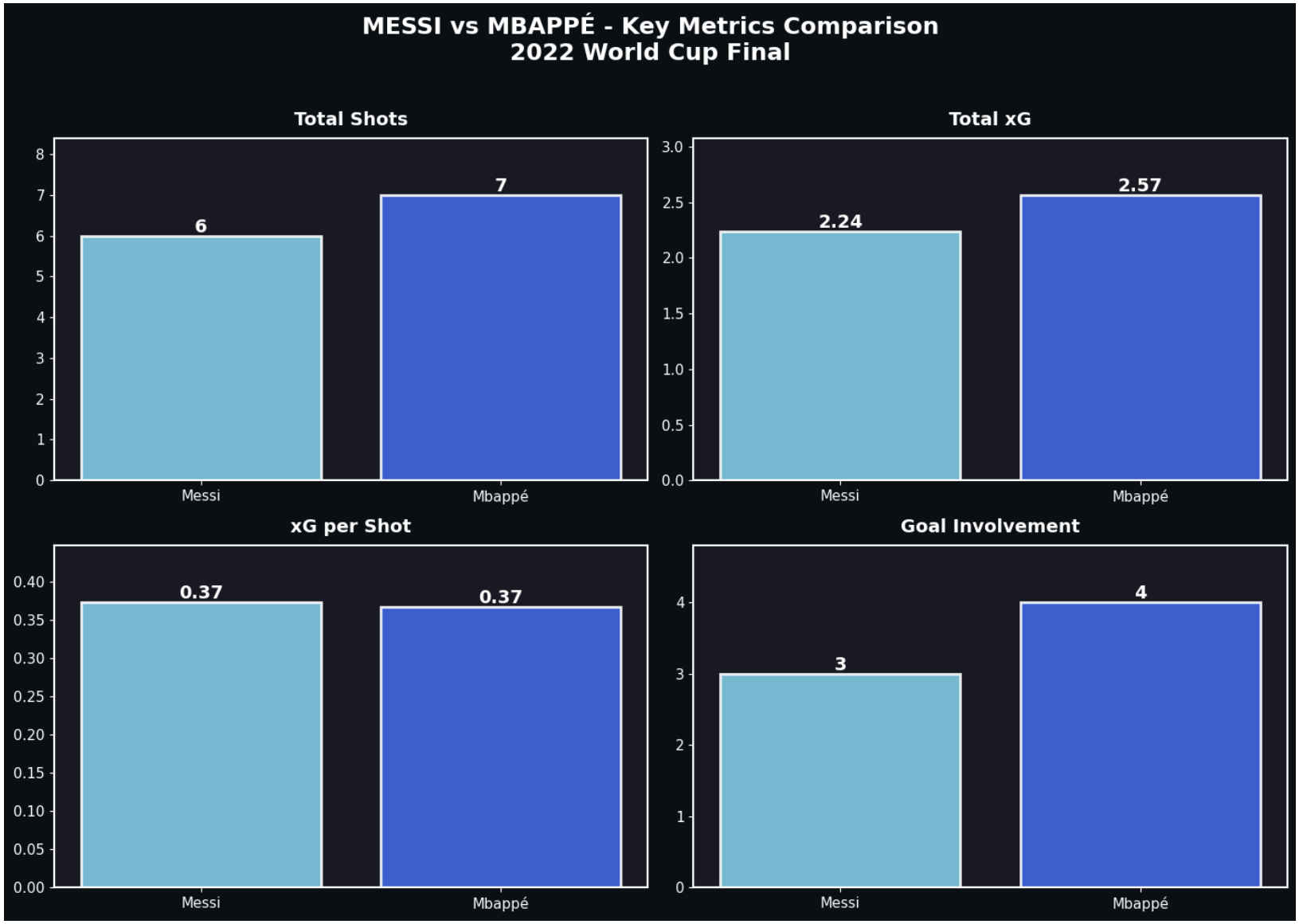


In [ ]:

In [32]:
```
# VISUALIZATION : Metrics Comparison Bar Chart - Messi Vs Mbappe

fig, axes = plt.subplots(2, 2, figsize=(14, 10), facecolor='#0e1117')
fig.suptitle(
    'MESSI vs MBAPPÉ - Key Metrics Comparison\n2022 World Cup Final',
    fontsize=18,
    weight='bold',
```

```
    color='white',
    y=0.98
)

metrics = [
    ('Total Shots', [messi_total_shots, mbappe_total_shots]),
    ('Total xG', [messi_total_xg, mbappe_total_xg]),
    ('xG per Shot', [messi_xg_per_shot, mbappe_xg_per_shot]),
    ('Goal Involvement', [int(messi_goals) + messi_assists, int(mbappe_goals) +
]

for idx, (ax, (metric_name, values)) in enumerate(zip(axes.flatten(), metrics)):
    ax.set_facecolor('#1a1d24')

    bars = ax.bar(
        ['Messi', 'Mbappé'],
        values,
        color=['#87CEEB', '#4169E1'],
        edgecolor='white',
        linewidth=2,
        alpha=0.9
    )

    # Add value labels on bars
    for bar, val in zip(bars, values):
        height = bar.get_height()
        ax.text(
            bar.get_x() + bar.get_width()/2.,
            height,
            f'{val:.2f}' if isinstance(val, float) else f'{val}',
            ha='center',
            va='bottom',
```

```
            fontsize=14,
            weight='bold',
            color='white'
        )

    ax.set_title(metric_name, fontsize=14, weight='bold', color='white', pad=10)
    ax.tick_params(colors='white', labelsize=11)
    ax.set_ylim(0, max(values) * 1.2)

    for spine in ax.spines.values():
        spine.set_edgecolor('white')
        spine.set_linewidth(1.5)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



In [ ]:

xG Timeline : Argentina VS France

Which Team Created Better Chances Throughout the Game?

In [34]:
```
# Prepare Shot Data for Both Teams

# Get all shots from the final
final_shots = events[events['type'] == 'Shot'].copy()
```

In [35]:
```
# Separate by team
argentina_shots = final_shots[final_shots['team'] == 'Argentina'].copy()
france_shots = final_shots[final_shots['team'] == 'France'].copy()

# Extract xG and minute data
argentina_shots = argentina_shots[['minute', 'team', 'player', 'shot_statsbomb_x
argentina_shots.rename(columns={'shot_statsbomb_xg': 'xg'}, inplace=True)
argentina_shots['goal'] = argentina_shots['shot_outcome'] == 'Goal'

france_shots = france_shots[['minute', 'team', 'player', 'shot_statsbomb_xg', 's
france_shots.rename(columns={'shot_statsbomb_xg': 'xg'}, inplace=True)
france_shots['goal'] = france_shots['shot_outcome'] == 'Goal'

# Sort by minute and calculate cumulative xG
argentina_shots = argentina_shots.sort_values('minute')
france_shots = france_shots.sort_values('minute')

argentina_shots['cum_xg'] = argentina_shots['xg'].cumsum()
france_shots['cum_xg'] = france_shots['xg'].cumsum()

print(f"Argentina shots: {len(argentina_shots)}")
print(f"France shots: {len(france_shots)}")
print(f"Argentina goals: {argentina_shots['goal'].sum()}")
print(f"France goals: {france_shots['goal'].sum()}")
```

```
Argentina shots: 24
France shots: 14
Argentina goals: 7
France goals: 5
```

In [36]:
```
france_shots.head()
```

Out[36]:

	minute	team	player	xg	shot_outcome	goal	cum_xg
4216	67	France	Randal Kolo Muani	0.096184	Off T	False	0.096184
4217	70	France	Kylian Mbappé Lottin	0.050644	Off T	False	0.146828
4219	79	France	Kylian Mbappé Lottin	0.783500	Goal	True	0.930328
4220	80	France	Kylian Mbappé Lottin	0.101703	Goal	True	1.032031
4222	92	France	Kylian Mbappé Lottin	0.032449	Blocked	False	1.064480

In [ ]:
```
```

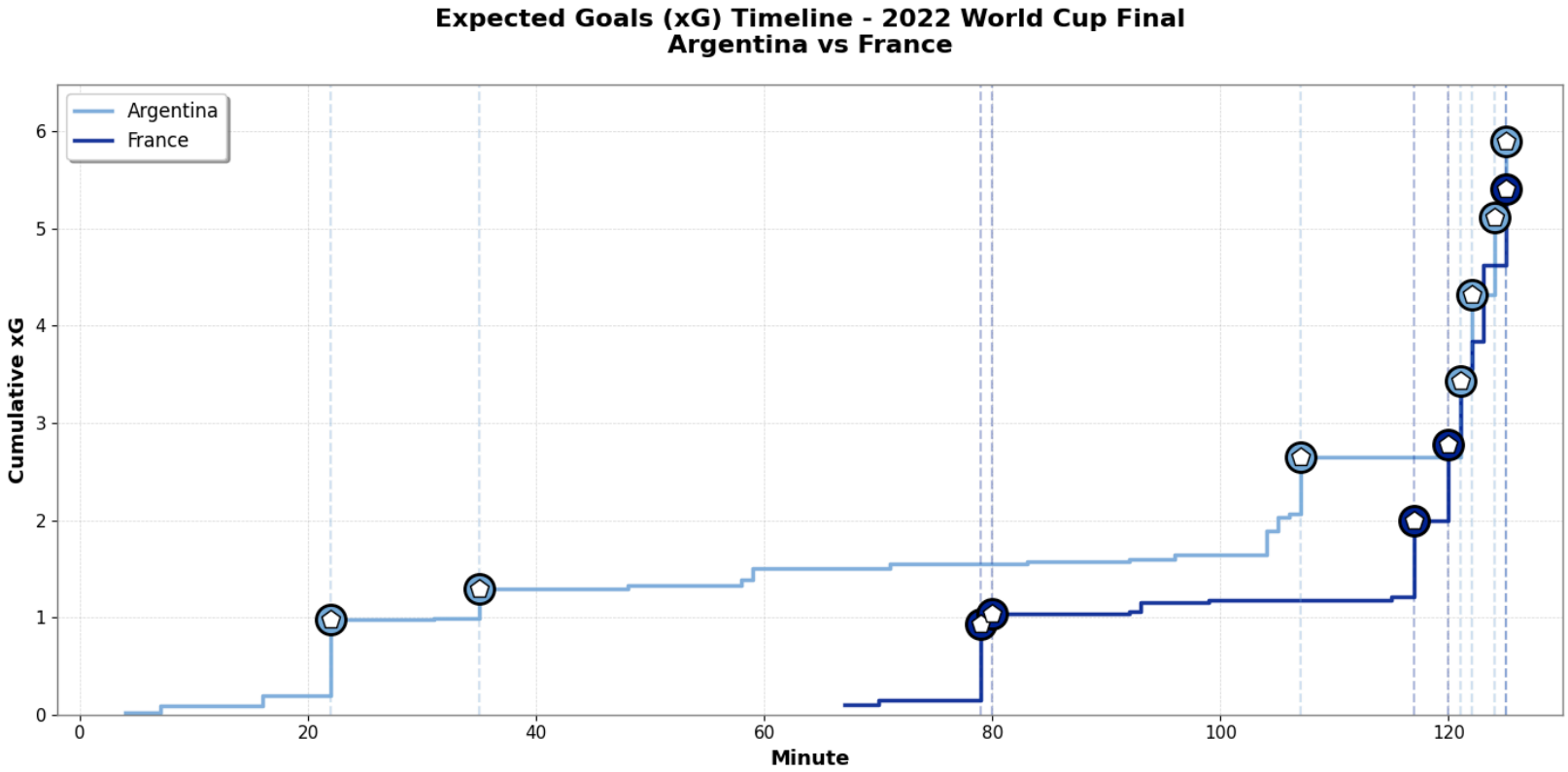
In [37]:
```
# Visualizing

fig, ax = plt.subplots(figsize=(14, 7), facecolor='white')
ax.set_facecolor('white')

# Team colors
argentina_color = '#75AADB'  # Light blue (Argentina)
france_color = '#002395'      # Navy blue (France)

# Plot step lines for cumulative xG
```

```
ax.step(
    argentina_shots['minute'],
    argentina_shots['cum_xg'],
    where='post',
    label='Argentina',
    linewidth=2.5,
    color=argentina_color,
    alpha=0.9
)

ax.step(
    france_shots['minute'],
    france_shots['cum_xg'],
    where='post',
    label='France',
    linewidth=2.5,
    color=france_color,
    alpha=0.9
)

# Mark goals with football markers (using large circles to simulate footballs)
argentina_goals = argentina_shots[argentina_shots['goal']]
france_goals = france_shots[france_shots['goal']]

# Argentina goals
ax.scatter(
    argentina_goals['minute'],
    argentina_goals['cum_xg'],
    s=350,
    color=argentina_color,
    edgecolors='black',
    linewidth=2,
```

```
    zorder=5,
    marker='o'
)

# Add white pattern to simulate football
for _, goal in argentina_goals.iterrows():
    ax.scatter(
        goal['minute'],
        goal['cum_xg'],
        s=150,
        color='white',
        edgecolors='black',
        linewidth=1,
        zorder=6,
        marker='p'
    )

# France goals
ax.scatter(
    france_goals['minute'],
    france_goals['cum_xg'],
    s=350,
    color=france_color,
    edgecolors='black',
    linewidth=2,
    zorder=5,
    marker='o'
)

# Add white pattern to simulate football
for _, goal in france_goals.iterrows():
    ax.scatter(
```

```
        goal['minute'],
        goal['cum_xg'],
        s=150,
        color='white',
        edgecolors='black',
        linewidth=1,
        zorder=6,
        marker='p'
    )

# Add vertical dashed lines for goals
for minute in argentina_goals['minute']:
    ax.axvline(minute, linestyle='--', alpha=0.3, color=argentina_color, linewid

for minute in france_goals['minute']:
    ax.axvline(minute, linestyle='--', alpha=0.3, color=france_color, linewidth=

# Styling
ax.set_xlabel('Minute', fontsize=13, weight='bold')
ax.set_ylabel('Cumulative xG', fontsize=13, weight='bold')
ax.set_title(
    'Expected Goals (xG) Timeline - 2022 World Cup Final\nArgentina vs France',
    fontsize=16,
    weight='bold',
    pad=20
)

# Legend
ax.legend(fontsize=12, loc='upper left', frameon=True, fancybox=True, shadow=Tru

# Grid
ax.grid(alpha=0.3, color='gray', linestyle='--', linewidth=0.5)
```

```
ax.tick_params(labelsize=11)

# Set spine colors
for spine in ax.spines.values():
    spine.set_edgecolor('gray')
    spine.set_linewidth(1)

# Set x-axis limits
ax.set_xlim(-2, max(argentina_shots['minute'].max(), france_shots['minute'].max(
ax.set_ylim(0, max(argentina_shots['cum_xg'].max(), france_shots['cum_xg'].max()

plt.tight_layout()
plt.show()
```



Overall Match Flow:

- Argentina dominated the first half with significantly higher xG accumulation, creating multiple quality chances before the 45th minute while France struggled to generate threatening opportunities.
- France's xG remained flat for most of regulation time until a dramatic surge from minute 80, indicating they created very few quality chances during the 90 minutes but came alive in the extended period.
- The steep rise in both teams' xG during extra time (minutes 90-120) shows the final period was end-to-end with both sides creating high-quality scoring opportunities, reflecting the dramatic 3-3 scoreline before penalties shootout.

In [ ]:

Team and Players Statistics Across The Entire 2022 World Cup Tournament

In [39]:
```
#team and player statistics across the entire 2022 World Cup tournament

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Rectangle
import matplotlib.patches as mpatches
```

In [40]:
```python
# Load All World Cup 2022 Matches

print("Loading all World Cup 2022 matches...")

# Get all match IDs from the tournament
all_match_ids = matches['match_id'].tolist()

print(f"Total matches in dataset: {len(all_match_ids)}")

# Load events for all matches (this may take a moment)
all_events = []
for i, match_id in enumerate(all_match_ids):
    print(f"Loading match {i+1}/{len(all_match_ids)}...", end='\r')
    match_events = sb.events(match_id=match_id)
    all_events.append(match_events)

# Combine all events
wc_events = pd.concat(all_events, ignore_index=True)
print(f"\nTotal events loaded: {len(wc_events)}")
```

```
Loading all World Cup 2022 matches...
Total matches in dataset: 64
Loading match 64/64...
Total events loaded: 234652
```

In [ ]:
```python

```

In [45]:
```python
#TEAM STATISTICS

print("\nCalculating team statistics...")
```

```python
# Initialize team stats dictionary
team_stats = []

for team in wc_events['team'].unique():
    team_events = wc_events[wc_events['team'] == team]

    # Possession (based on pass count as proxy)
    total_passes = len(team_events[team_events['type'] == 'Pass'])

    # Shots
    shots = team_events[team_events['type'] == 'Shot']
    total_shots = len(shots)
    shots_on_target = len(shots[shots['shot_outcome'].isin(['Goal', 'Saved'])])

    # xG
    total_xg = shots['shot_statsbomb_xg'].sum()

    # Goals
    goals = len(shots[shots['shot_outcome'] == 'Goal'])

    # Assists
    assists = len(team_events[team_events['pass_goal_assist'] == True])

    # Pass completion
    successful_passes = len(team_events[
        (team_events['type'] == 'Pass') &
        (team_events['pass_outcome'].isna())
    ])
    pass_completion = (successful_passes / total_passes * 100) if total_passes >

    # Tackles and Interceptions
    tackles = len(team_events[team_events['type'] == 'Tackle'])
```

```
    interceptions = len(team_events[team_events['type'] == 'Interception'])

    team_stats.append({
        'Team': team,
        'Passes': total_passes,
        'Pass %': pass_completion,
        'Shots': total_shots,
        'Shots on Target': shots_on_target,
        'xG': total_xg,
        'Goals': goals,
        'Assists': assists,
        'Tackles': tackles,
        'Interceptions': interceptions
    })

teams_df = pd.DataFrame(team_stats)
```

```
Calculating team statistics...
```

In [46]:
```
teams_df.head()
```

Out[46]:

	Team	Passes	Pass %	Shots	Shots on Target	xG	Goals	Assists	Tackles	Interceptior
0	Serbia	1503	78.908849	32	9	3.090398	5	4	0	2
1	Switzerland	2035	82.211302	37	13	6.090516	5	4	0	4
2	Argentina	4615	85.352113	110	56	20.988443	23	8	0	7
3	Australia	1707	74.458114	26	8	1.578172	3	2	0	4
4	Denmark	1928	83.661826	35	11	3.192257	1	0	0	3

In [ ]:

In [47]:
```
# PLAYER STATISTICS

print("Calculating player statistics...")

player_stats = []

for player in wc_events['player'].dropna().unique():
    player_events = wc_events[wc_events['player'] == player]
    team = player_events['team'].iloc[0] if len(player_events) > 0 else 'Unknown

    # Shots
    shots = player_events[player_events['type'] == 'Shot']
    total_shots = len(shots)

    # xG
```

```
    total_xg = shots['shot_statsbomb_xg'].sum()

    # Goals
    goals = len(shots[shots['shot_outcome'] == 'Goal'])

    # Assists
    assists = len(player_events[player_events['pass_goal_assist'] == True])

    # Passes
    passes = player_events[player_events['type'] == 'Pass']
    total_passes = len(passes)
    successful_passes = len(passes[passes['pass_outcome'].isna()])
    pass_completion = (successful_passes / total_passes * 100) if total_passes >

    # Key passes (passes leading to shots)
    key_passes = len(player_events[player_events['pass_shot_assist'] == True])

    # Dribbles
    dribbles = len(player_events[player_events['type'] == 'Dribble'])

    player_stats.append({
        'Player': player,
        'Team': team,
        'Goals': goals,
        'Assists': assists,
        'xG': total_xg,
        'Shots': total_shots,
        'Passes': total_passes,
        'Pass %': pass_completion,
        'Key Passes': key_passes,
        'Dribbles': dribbles
    })
```

```
players_df = pd.DataFrame(player_stats)

print("Data preparation complete!")
```

```
Calculating player statistics...
Data preparation complete!
```

In [49]:
```
players_df.head(5)
```

Out[49]:

	Player	Team	Goals	Assists	xG	Shots	Passes	Pass %	Key Passes	Dribbles
0	Breel-Donald Embolo	Switzerland	2	0	2.391197	6	66	75.757576	2	4
1	Remo Freuler	Switzerland	1	0	0.544795	3	149	84.563758	0	4
2	Silvan Widmer	Switzerland	0	1	0.051174	1	164	77.439024	1	0
3	Fabian Lukas Schär	Switzerland	0	0	0.103673	2	80	80.000000	0	0
4	Djibril Sow	Switzerland	0	1	0.150380	2	96	90.625000	2	1

In [ ]:

In [51]:

```
# ==========================================
# TOP 5 TEAMS - OVERALL PERFORMANCE
# ==========================================

# Prepare data using teams_df dataframe
top_teams = (teams_df[['Team', 'Shots', 'xG', 'Goals']]
             .sort_values('Goals', ascending=False)
             .head(5)
             .reset_index(drop=True))

# Style the table
styled_teams = (top_teams.style
    .format({'xG': '{:.2f}'})  # Format xG to 2 decimals
    .bar(subset=['Shots'], color='#4e79a7')  # Blue bars for shots
    .bar(subset=['Goals'], color='#e15759')  # Red bars for goals
    .background_gradient(subset=['xG'], cmap='Greens')  # Green gradient for xG
    .set_caption('Top 5 Teams by xG - 2022 FIFA World Cup')
    .set_table_styles([
        {'selector': 'caption', 'props': [('font-size', '16px'), ('font-weight',
    ]))

display(styled_teams)
```

Top 5 Teams by xG - 2022 FIFA World Cup

	Team	Shots	xG	Goals
0	Argentina	110	20.99	23
1	France	106	14.96	18
2	Croatia	87	13.17	15
3	Netherlands	48	8.91	13
4	England	63	8.74	13

- The top 5 teams all scored more goals than their xG predicted, showing that top ranked teams possess superior finishing quality that exceeds statistical expectations.
- The table also reveals Argentina's tournament dominance, leading in shot volume (110), quality (20.99 xG) and goals (23).
- France demonstrated superior finishing efficiency, scoring 18 goals from just 14.96 expected goals.

In [ ]:

In [52]:
```
# ==========================================
# TOP 10 PASSERS
# ==========================================
```

```python
# Prepare data (filter players with min 50 passes)
top_passers = (players_df[players_df['Passes'] >= 50]
               [['Player', 'Team', 'Passes', 'Pass %']]
               .sort_values('Passes', ascending=False)
               .head(10)
               .reset_index(drop=True))

# Style the table
styled_passers = (top_passers.style
    .format({'Pass %': '{:.1f}%'})  # Format pass % to 1 decimal
    .bar(subset=['Passes'], color='#4e79a7')  # Blue bars for passes
    .background_gradient(subset=['Pass %'], cmap='Greens')  # Green gradient for
    .set_caption('Top 10 Passers (min. 50 passes)')
    .set_table_styles([
        {'selector': 'caption', 'props': [('font-size', '16px'), ('font-weight',
    ]))

display(styled_passers)
```

Top 10 Passers (min. 50 passes)

	Player	Team	Passes	Pass %
0	Rodrigo Hernández Cascante	Spain	677	94.8%
1	Nicolás Hernán Otamendi	Argentina	562	93.4%
2	Rodrigo Javier De Paul	Argentina	562	84.3%
3	Luka Modrić	Croatia	547	84.3%
4	Marcelo Brozović	Croatia	528	89.2%
5	Joško Gvardiol	Croatia	508	90.9%
6	Enzo Fernandez	Argentina	469	88.5%
7	Aurélien Djani Tchouaméni	France	468	92.1%
8	Mateo Kovačić	Croatia	454	91.4%
9	Dejan Lovren	Croatia	452	89.6%

In [ ]:

In [53]:
```
# ==========================================
# TOP 10 ASSISTS BY PLAYERS
# ==========================================

top_assists = (players_df[['Player', 'Team', 'Assists', 'Key Passes']]
               .sort_values('Assists', ascending=False)
               .head(10)
               .reset_index(drop=True))
```

```
# Style the table
styled_assists = (top_assists.style
    .bar(subset=['Assists'], color='#e15759')  # Red bars for assists
    .bar(subset=['Key Passes'], color='#f28e2b')  # Orange bars for key passes
    .set_caption('Top 10 Assist Providers')
    .set_table_styles([
        {'selector': 'caption', 'props': [('font-size', '16px'), ('font-weight',
    ]))

display(styled_assists)
```

Top 10 Assist Providers

	Player	Team	Assists	Key Passes
0	Lionel Andrés Messi Cuccittini	Argentina	3	16
1	Harry Kane	England	3	2
2	Bruno Miguel Borges Fernandes	Portugal	3	6
3	Antoine Griezmann	France	3	17
4	Ivan Perišić	Croatia	3	6
5	Dušan Tadić	Serbia	2	7
6	Mislav Oršić	Croatia	2	3
7	João Félix Sequeira	Portugal	2	1
8	Raphaël Adelino José Guerreiro	Portugal	2	4
9	Vinícius José Paixão de Oliveira Júnior	Brazil	2	6

In [ ]:

AVG POSSESSION & PASSING PER MATCH

In [54]:
```
# ========================================
# AVG POSSESSION & PASSING PER MATCH
# ========================================

# Calculate per-match statistics
```

```
team_match_stats = []

for team in wc_events['team'].unique():
    team_events = wc_events[wc_events['team'] == team]

    # Get all matches this team played
    team_match_ids = team_events['match_id'].unique()

    for match_id in team_match_ids:
        # Get all events from this specific match
        match_events = wc_events[wc_events['match_id'] == match_id]

        # Team's passes in this match
        team_passes_in_match = len(match_events[
            (match_events['team'] == team) &
            (match_events['type'] == 'Pass')
        ])

        # Total passes in this match (both teams)
        total_passes_in_match = len(match_events[match_events['type'] == 'Pass']

        # Possession proxy for this match
        possession_pct = (team_passes_in_match / total_passes_in_match * 100) if

        # Pass accuracy for this match
        team_match_passes = match_events[
            (match_events['team'] == team) &
            (match_events['type'] == 'Pass')
        ]
        successful = len(team_match_passes[team_match_passes['pass_outcome'].isn
        accuracy = (successful / team_passes_in_match * 100) if team_passes_in_m
```

```
        team_match_stats.append({
            'Team': team,
            'Match_ID': match_id,
            'Possession %': possession_pct,
            'Passes': team_passes_in_match,
            'Pass Accuracy %': accuracy
        })

# Convert to DataFrame
match_stats_df = pd.DataFrame(team_match_stats)

# Calculate averages per team
team_averages = match_stats_df.groupby('Team').agg({
    'Possession %': 'mean',
    'Passes': 'mean',
    'Pass Accuracy %': 'mean',
    'Match_ID': 'count'
}).reset_index()

team_averages.rename(columns={
    'Possession %': 'Avg Possession %',
    'Passes': 'Avg Passes',
    'Pass Accuracy %': 'Avg Pass Accuracy %',
    'Match_ID': 'Matches Played'
}, inplace=True)

# Top 10 by possession
top_possession = (team_averages[['Team', 'Avg Possession %', 'Avg Passes', 'Avg
                          .sort_values('Avg Possession %', ascending=False)
                          .head(10)
                          .reset_index(drop=True))
```

```
# Style the table
styled_possession = (top_possession.style
    .format({
        'Avg Possession %': '{:.1f}%',
        'Avg Passes': '{:.1f}',
        'Avg Pass Accuracy %': '{:.1f}%'
    })
    .bar(subset=['Avg Possession %'], color='#59a14f')
    .bar(subset=['Avg Passes'], color='#4e79a7')
    .background_gradient(subset=['Avg Pass Accuracy %'], cmap='Greens')
    .set_caption('Top 10 Teams - Possession & Passes')
    .set_table_styles([
        {'selector': 'caption', 'props': [('font-size', '16px'), ('font-weight',
    ]))

display(styled_possession)
```

Top 10 Teams - Possession & Passes

	Team	Avg Possession %	Avg Passes	Avg Pass Accuracy %	Matches Played
0	Spain	75.3%	978.5	90.1%	4
1	England	63.1%	647.6	86.3%	5
2	Portugal	60.9%	625.8	84.8%	5
3	Denmark	59.7%	642.7	83.7%	3
4	Germany	58.9%	654.3	84.4%	3
5	Argentina	57.7%	659.3	84.9%	7
6	Belgium	56.8%	622.3	85.5%	3
7	Brazil	56.0%	639.2	86.8%	5
8	Croatia	54.3%	644.4	84.0%	7
9	Mexico	53.9%	478.3	77.0%	3

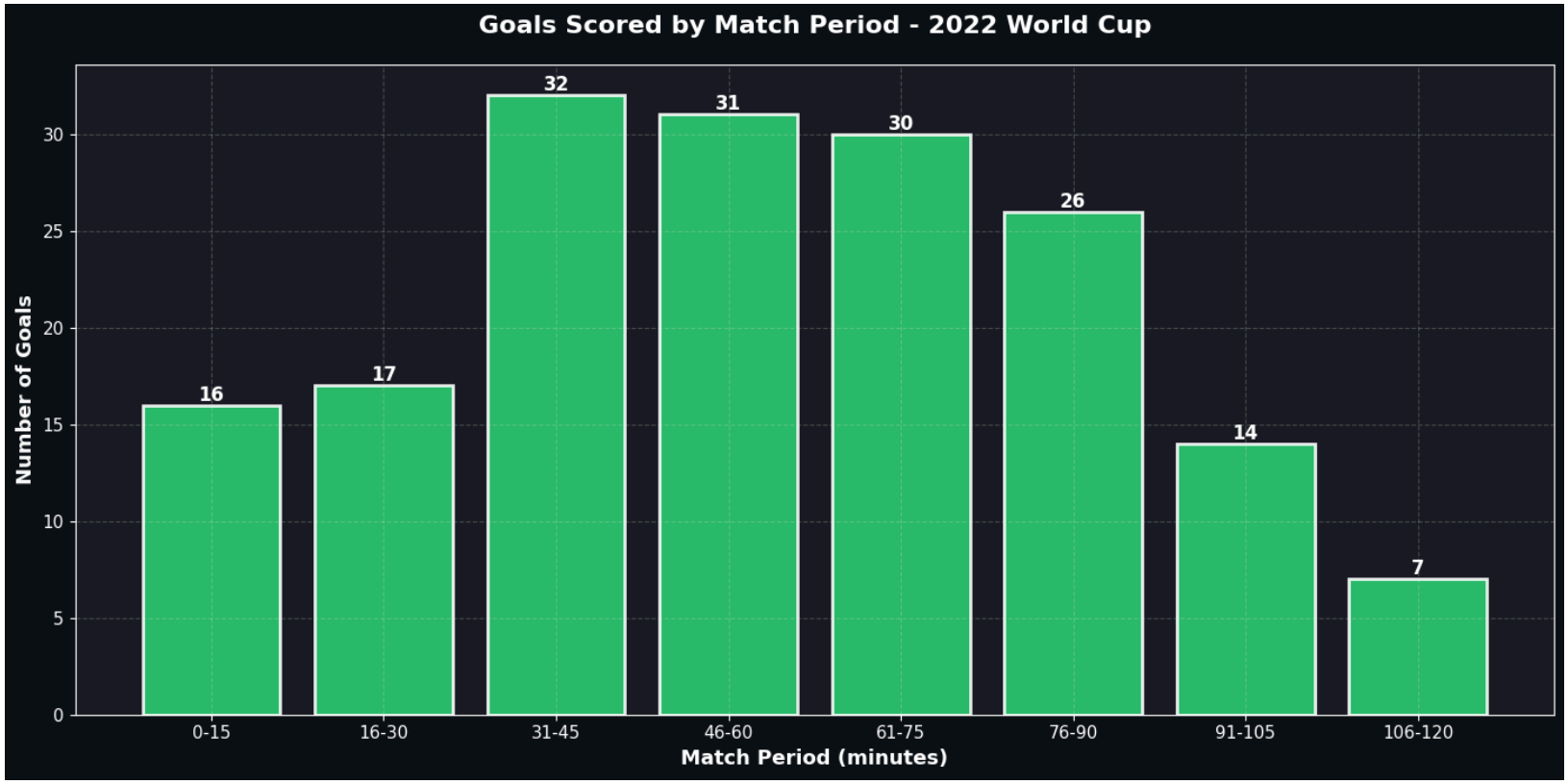
In [ ]:

When Do Goals Happen Most? Scoring Patterns Across Match Periods.

In [55]:
```
# ============================================
# TOURNAMENT-WIDE: GOALS BY MATCH MINUTE
```

```python
# ==========================================

def goals_by_minute_tournament(events_df):
    """When are goals scored in the tournament"""
    goals = events_df[
        (events_df['type'] == 'Shot') &
        (events_df['shot_outcome'] == 'Goal')
    ].copy()

    # Bin by 15-minute periods
    goals['period_bin'] = pd.cut(goals['minute'],
                                 bins=[0, 15, 30, 45, 60, 75, 90, 105, 120],
                                 labels=['0-15', '16-30', '31-45', '46-60',
                                         '61-75', '76-90', '91-105', '106-120'])

    period_counts = goals['period_bin'].value_counts().sort_index()

    fig, ax = plt.subplots(figsize=(14, 7), facecolor='#0e1117')
    ax.set_facecolor('#1a1d24')

    bars = ax.bar(period_counts.index, period_counts.values,
                  color='#2ecc71', edgecolor='white', linewidth=2, alpha=0.9)

    # Highlight values on bars
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{int(height)}', ha='center', va='bottom',
                fontsize=12, weight='bold', color='white')

    ax.set_title('Goals Scored by Match Period - 2022 World Cup',
                 fontsize=16, weight='bold', color='white', pad=20)
```

```
    ax.set_xlabel('Match Period (minutes)', fontsize=13, color='white', weight='
    ax.set_ylabel('Number of Goals', fontsize=13, color='white', weight='bold')
    ax.tick_params(colors='white', labelsize=11)
    ax.grid(alpha=0.2, color='white', linestyle='--')

    for spine in ax.spines.values():
        spine.set_edgecolor('white')

    plt.tight_layout()
    return fig

fig_goals_time = goals_by_minute_tournament(wc_events)
plt.show()
```



Key Insights:

- Goals peak just before halftime (31–45 minutes), suggesting teams increase attacking intensity as the first half draws to a close.
- The early second half (46–75 minutes) remains highly productive for goals scored, reflecting tactical adjustments and increased tempo after halftime.
- Goal output drops sharply in extra time, indicating fatigue, conservative play, and higher risk management in knockout scenarios.

In [ ]:

Which Teams Had the Best Conversion Rate?

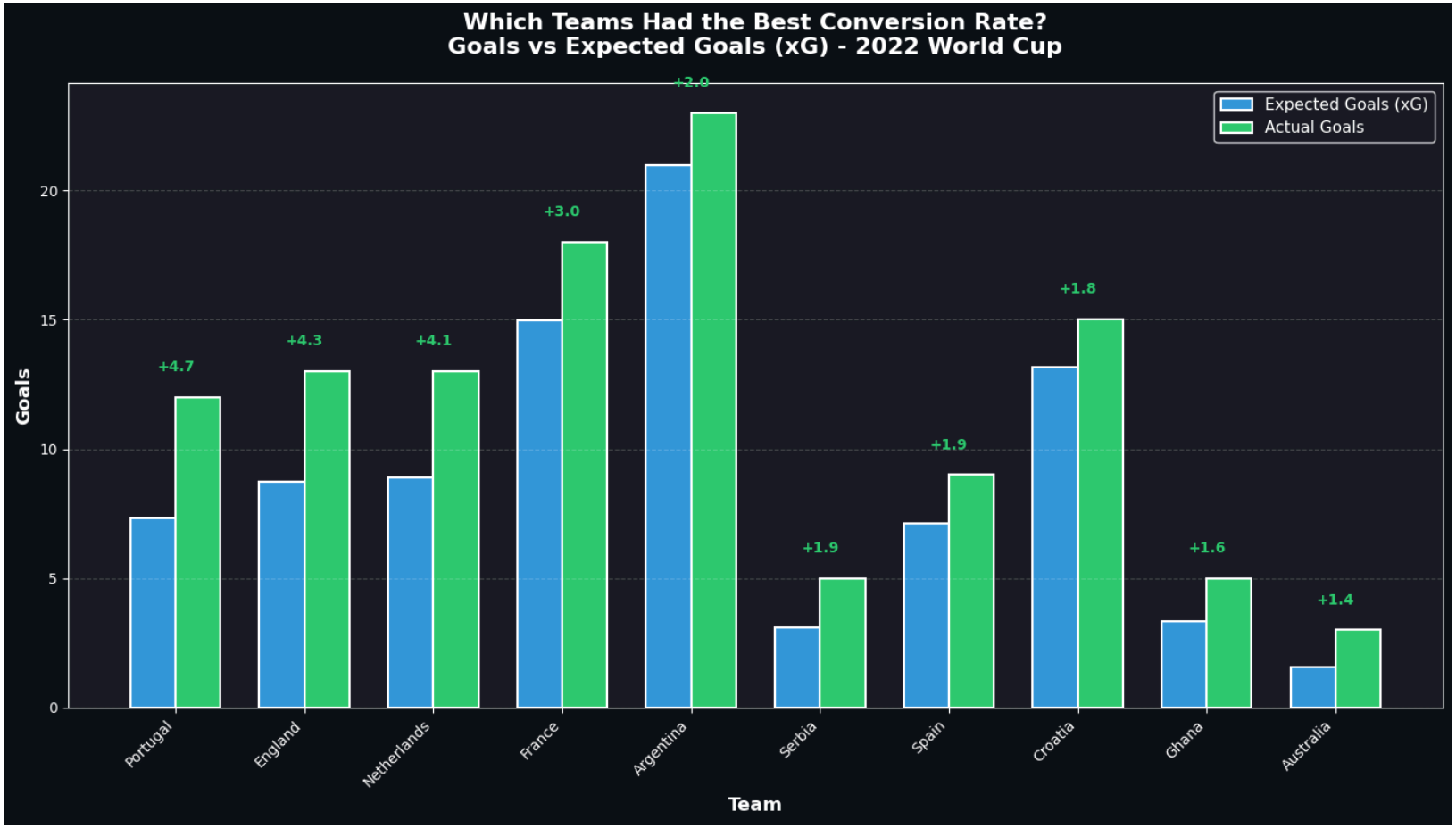
In [55]:
```
# ============================================
# 9. WHICH TEAMS HAD THE BEST CONVERSION RATE?
#    (Goals vs xG Performance)
# ============================================

def conversion_efficiency_analysis(events_df):
    """Analyze which teams overperformed/underperformed their xG"""

    team_conversion = []

    for team in events_df['team'].unique():
        team_shots = events_df[
```

```
            (events_df['team'] == team) &
            (events_df['type'] == 'Shot')
        ]

        total_xg = team_shots['shot_statsbomb_xg'].sum()
        actual_goals = len(team_shots[team_shots['shot_outcome'] == 'Goal'])

        conversion_diff = actual_goals - total_xg
        conversion_rate = (actual_goals / len(team_shots) * 100) if len(team_sho

        team_conversion.append({
            'Team': team,
            'xG': total_xg,
            'Goals': actual_goals,
            'Difference': conversion_diff,
            'Conversion %': conversion_rate
        })

    conversion_df = pd.DataFrame(team_conversion).sort_values('Difference', asce

    # Visualize top 10
    top_10 = conversion_df.head(10)

    fig, ax = plt.subplots(figsize=(14, 8), facecolor='#0e1117')
    ax.set_facecolor('#1a1d24')

    x = np.arange(len(top_10))
    width = 0.35

    bars1 = ax.bar(x - width/2, top_10['xG'], width,
                   label='Expected Goals (xG)', color='#3498db',
                   edgecolor='white', linewidth=1.5)
```

```
    bars2 = ax.bar(x + width/2, top_10['Goals'], width,
                   label='Actual Goals', color='#2ecc71',
                   edgecolor='white', linewidth=1.5)

    # Add difference annotations
    for i, (xg, goals, diff) in enumerate(zip(top_10['xG'], top_10['Goals'], top
        color = '#2ecc71' if diff > 0 else '#e74c3c'
        ax.text(i, max(xg, goals) + 1, f'{diff:+.1f}',
                ha='center', fontsize=10, weight='bold', color=color)

    ax.set_xlabel('Team', fontsize=13, color='white', weight='bold')
    ax.set_ylabel('Goals', fontsize=13, color='white', weight='bold')
    ax.set_title('Which Teams Had the Best Conversion Rate?\nGoals vs Expected G
                 fontsize=16, weight='bold', color='white', pad=20)
    ax.set_xticks(x)
    ax.set_xticklabels(top_10['Team'], rotation=45, ha='right', color='white')
    ax.legend(fontsize=11, facecolor='#1a1d24', edgecolor='white', labelcolor='w
    ax.tick_params(colors='white')
    ax.grid(alpha=0.2, color='white', linestyle='--', axis='y')

    for spine in ax.spines.values():
        spine.set_edgecolor('white')

    plt.tight_layout()

    print("\n" + "="*60)
    print("TOP 5 TEAMS - CONVERSION EFFICIENCY")
    print("="*60)
    for idx, row in conversion_df.head(5).iterrows():
        print(f"{row['Team']:15} | xG: {row['xG']:5.2f} | Goals: {row['Goals']:2
    print("="*60)
```

```
    return fig

fig_conversion = conversion_efficiency_analysis(wc_events)
plt.show()
```

```
============================================================
TOP 5 TEAMS - CONVERSION EFFICIENCY
============================================================
Portugal        | xG:  7.31 | Goals: 12 | Diff: +4.69
England         | xG:  8.74 | Goals: 13 | Diff: +4.26
Netherlands     | xG:  8.91 | Goals: 13 | Diff: +4.09
France          | xG: 14.96 | Goals: 18 | Diff: +3.04
Argentina       | xG: 20.99 | Goals: 23 | Diff: +2.01
============================================================
```



Key Insights:

- Portugal show the strongest overperformance relative to xG, indicating exceptional finishing efficiency compared to chance quality. England is 2nd with xG of +4.3.
- Argentina and France both exceed their xG totals, combining high chance creation with clinical conversion at the tournament's highest level.

- Several teams outperform xG by smaller margins, suggesting effective finishing but less consistent chance volume or shot quality.

In [ ]:

Where Do Most Goals Come From?

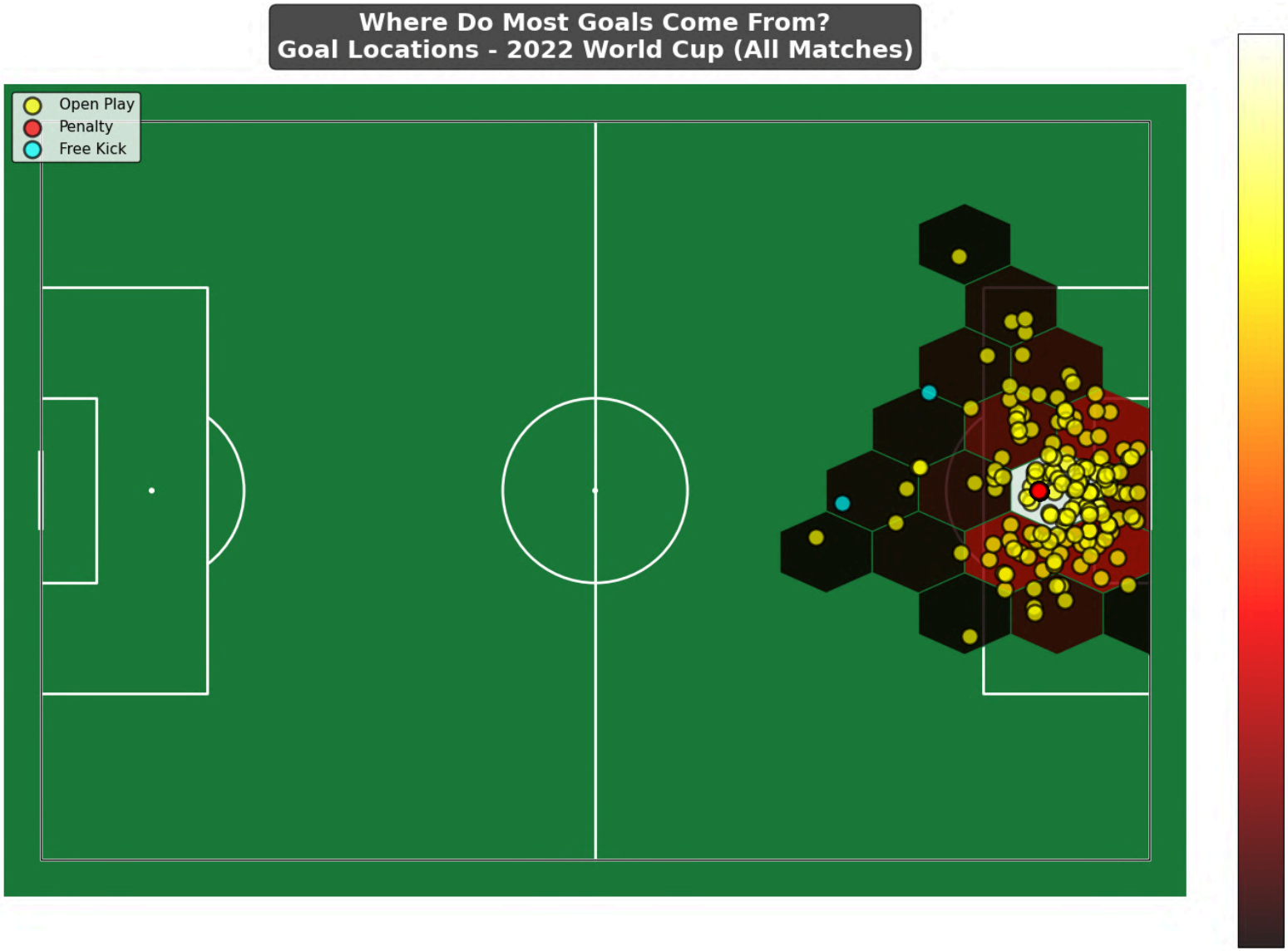
Shot Location Heatmap for All Goals

In [56]:
```
# ============================================
# 10. WHERE DO MOST GOALS COME FROM?
#     (Shot Location Heatmap for All Goals)
# ============================================

def goal_location_heatmap(events_df):
    """Analyze from where goals were scored in the tournament"""

    goals = events_df[
        (events_df['type'] == 'Shot') &
        (events_df['shot_outcome'] == 'Goal')
    ].copy()

    goals['x'] = goals['location'].str[0]
    goals['y'] = goals['location'].str[1]

    # Categorize by type
    goals['type_category'] = 'Open Play'
    goals.loc[goals['shot_type'] == 'Penalty', 'type_category'] = 'Penalty'
    goals.loc[goals['shot_type'] == 'Free Kick', 'type_category'] = 'Free Kick'
```

```python
# Create visualization
pitch = Pitch(pitch_type="statsbomb", pitch_color="#1b7a3a",
              line_color="white", linewidth=2)
fig, ax = plt.subplots(figsize=(14, 10), facecolor='white')
pitch.draw(ax=ax)

# Hexbin for goal density
hexmap = pitch.hexbin(goals['x'], goals['y'],
                      gridsize=12, cmap='hot',
                      edgecolors='#1b7a3a', linewidth=1,
                      ax=ax, alpha=0.85, mincnt=1)

# Add colorbar
cbar = plt.colorbar(hexmap, ax=ax, fraction=0.046, pad=0.04)
cbar.set_label('Number of Goals', color='white', fontsize=12, weight='bold')
cbar.ax.tick_params(colors='white')

# Add goal markers
colors = {'Open Play': 'yellow', 'Penalty': 'red', 'Free Kick': 'cyan'}
for goal_type, color in colors.items():
    type_goals = goals[goals['type_category'] == goal_type]
    if len(type_goals) > 0:
        pitch.scatter(type_goals['x'], type_goals['y'],
                      s=150, color=color, edgecolors='black',
                      linewidth=2, alpha=0.7, label=goal_type, ax=ax, zorder=

ax.legend(fontsize=11, loc='upper left', facecolor='white', edgecolor='black
ax.set_title('Where Do Most Goals Come From?\nGoal Locations - 2022 World Cu
             fontsize=18, weight='bold', color='white', pad=20,
             bbox=dict(boxstyle='round', facecolor='black', alpha=0.7))
```

```
    plt.tight_layout()

    # Statistics
    print("\n" + "="*60)
    print("GOAL LOCATION STATISTICS")
    print("="*60)
    print(f"Total Goals: {len(goals)}")
    print(f"\nBy Type:")
    print(goals['type_category'].value_counts())
    print(f"\nAverage Goal Distance from Goal: {(120 - goals['x']).mean():.1f} y
    print("="*60)

    return fig

fig_goal_locations = goal_location_heatmap(wc_events)
plt.show()
```

```
============================================================
GOAL LOCATION STATISTICS
============================================================
Total Goals: 195

By Type:
type_category
Open Play    150
Penalty       43
Free Kick      2
Name: count, dtype: int64

Average Goal Distance from Goal: 10.8 yards
============================================================
```



- The overwhelming majority of goals were scored from inside the penalty area from open play, particularly in central positions near the six-yard box.

- Penalties and free kicks constituted a substantial minority of total goals, highlighting set piece proficiency as a critical tournament success factor.

In [ ]:

How Effective Were Set Pieces?

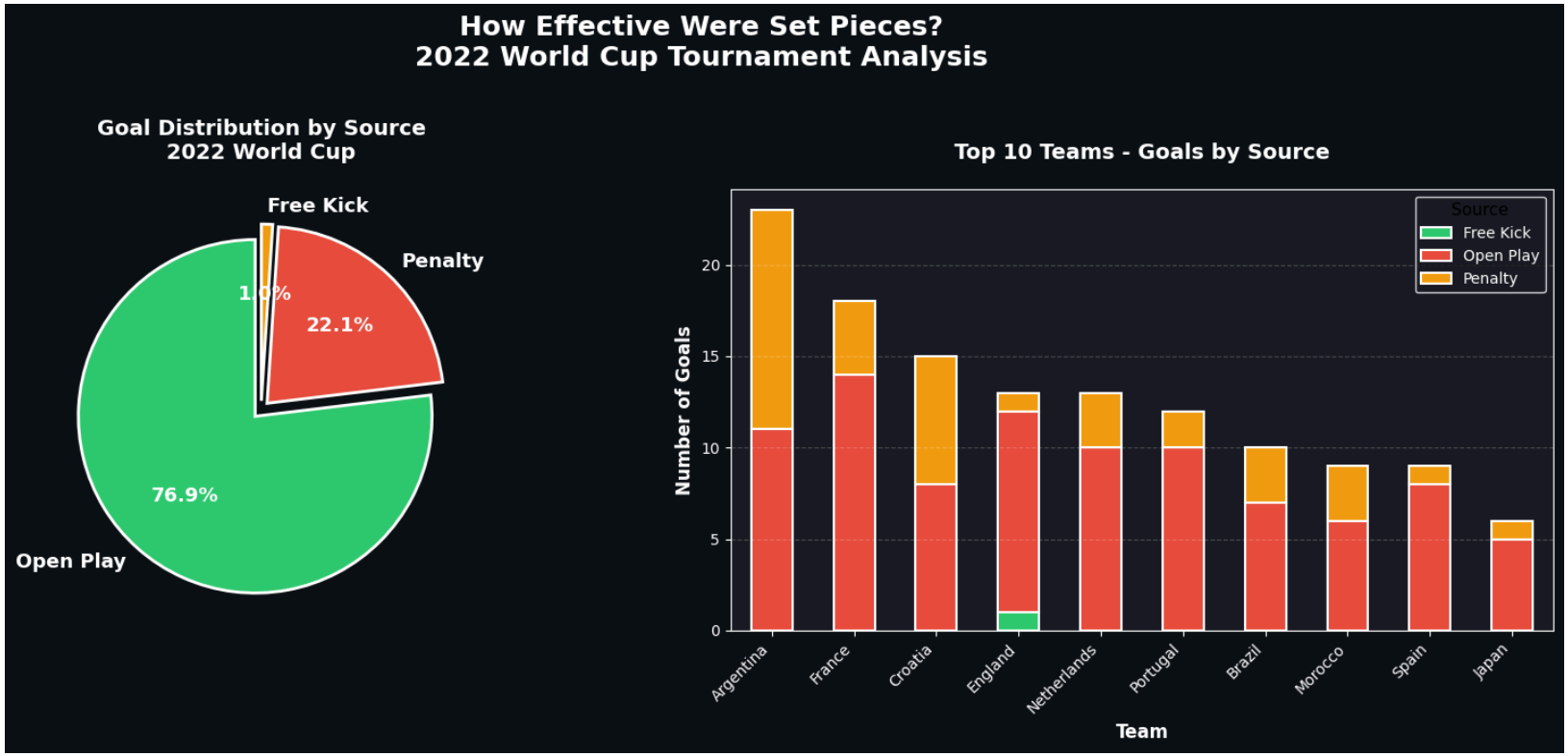
(Goals from Set Pieces vs Open Play)

In [56]:
```
# ============================================
# 12. HOW EFFECTIVE WERE SET PIECES?
#     (Goals from Set Pieces vs Open Play)
# ============================================

def set_piece_effectiveness(events_df):
    """Analyze goals from set pieces vs open play"""

    goals = events_df[
        (events_df['type'] == 'Shot') &
        (events_df['shot_outcome'] == 'Goal')
    ].copy()

    # Categorize goal types
    goals['source'] = 'Open Play'
    goals.loc[goals['shot_type'] == 'Penalty', 'source'] = 'Penalty'
    goals.loc[goals['shot_type'] == 'Free Kick', 'source'] = 'Free Kick'
```

```
    # Create visualization - pie chart and bar chart
    fig, axes = plt.subplots(1, 2, figsize=(16, 7), facecolor='#0e1117')

    # Pie chart
    source_counts = goals['source'].value_counts()
    colors = ['#2ecc71', '#e74c3c', '#f39c12']
    explode = (0.05, 0.05, 0.05)

    axes[0].pie(source_counts.values, labels=source_counts.index,
                autopct='%1.1f%%', startangle=90, colors=colors,
                explode=explode, textprops={'fontsize': 13, 'weight': 'bold', 'co
                wedgeprops={'edgecolor': 'white', 'linewidth': 2})
    axes[0].set_title('Goal Distribution by Source\n2022 World Cup',
                      fontsize=14, weight='bold', color='white', pad=20)
    axes[0].set_facecolor('#1a1d24')

    # Bar chart by team
    team_goal_sources = goals.groupby(['team', 'source']).size().unstack(fill_va
    team_totals = team_goal_sources.sum(axis=1).sort_values(ascending=False).hea
    top_teams_goals = team_goal_sources.loc[team_totals.index]

    top_teams_goals.plot(kind='bar', stacked=True, ax=axes[1],
                         color=colors, edgecolor='white', linewidth=1.5)

    axes[1].set_title('Top 10 Teams - Goals by Source',
                      fontsize=14, weight='bold', color='white', pad=20)
    axes[1].set_xlabel('Team', fontsize=12, color='white', weight='bold')
    axes[1].set_ylabel('Number of Goals', fontsize=12, color='white', weight='bo
    axes[1].tick_params(colors='white')
    axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=45, ha='right',
    axes[1].legend(title='Source', fontsize=10, title_fontsize=11,
                   facecolor='#1a1d24', edgecolor='white', labelcolor='white')
```

```
    axes[1].grid(alpha=0.2, color='white', linestyle='--', axis='y')
    axes[1].set_facecolor('#1a1d24')

    for spine in axes[1].spines.values():
        spine.set_edgecolor('white')

    fig.suptitle('How Effective Were Set Pieces?\n2022 World Cup Tournament Anal
                 fontsize=18, weight='bold', color='white', y=0.98)

    plt.tight_layout(rect=[0, 0, 1, 0.96])

    print("\n" + "="*60)
    print("SET PIECE EFFECTIVENESS")
    print("="*60)
    print(f"Total Goals: {len(goals)}")
    print(f"\nBreakdown:")
    for source, count in source_counts.items():
        percentage = (count / len(goals) * 100)
        print(f"{source:15} | {count:3} goals ({percentage:5.1f}%)")
    print("="*60)

    return fig

fig_set_pieces = set_piece_effectiveness(wc_events)
plt.show()
```

```
============================================================
SET PIECE EFFECTIVENESS
============================================================
Total Goals: 195

Breakdown:
Open Play       | 150 goals ( 76.9%)
Penalty         |  43 goals ( 22.1%)
Free Kick       |   2 goals (  1.0%)
============================================================
```



- Nearly 77% of World Cup goals came from open play

- Penalties accounted for 22.1% of all goals, making penalty-area incursions and defensive discipline critical tournament factors.

In [ ]:

Uzoh C. Hillary

LinkedIn : http://www.linkedin.com/in/hillaryuzoh

Github : https://github.com/Uzo-Hill

Email : uzohhillary@gmail.com

In [ ]: